

Web Application Penetration Testing Report

Target Application: Damn Vulnerable Web Application (DVWA)

Scope: Reflected XSS, Stored XSS, DOM-Based XSS, and Blind SQL Injection

Executive Summary

During the security assessment of the DVWA application, multiple web application vulnerabilities were identified across different security levels (Low and Medium). These vulnerabilities allow potential attackers to execute malicious JavaScript code in users' browsers and send unauthorized commands to the backend database.

Detailed Vulnerability Findings

1. Reflected Cross-Site Scripting (XSS)

Severity / Level: Low & Medium

Description: The web application receives user input from the URL or input forms and displays it immediately on the page without proper sanitization or encoding.

Finding (Low): The application reads input directly from the name parameter in the URL. Injecting `<script>alert('XSS')</script>` runs instantly.

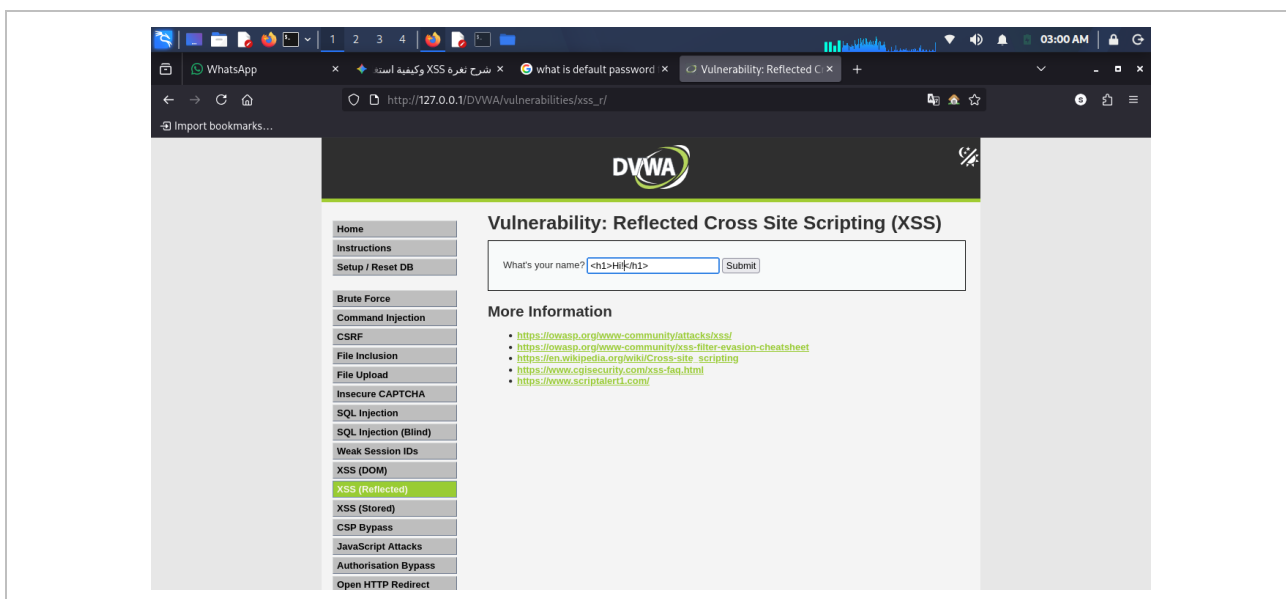


Figure 1.1 – Crafting a baseline HTML injection test (`<h1>Hi!</h1>`) in the name input field.

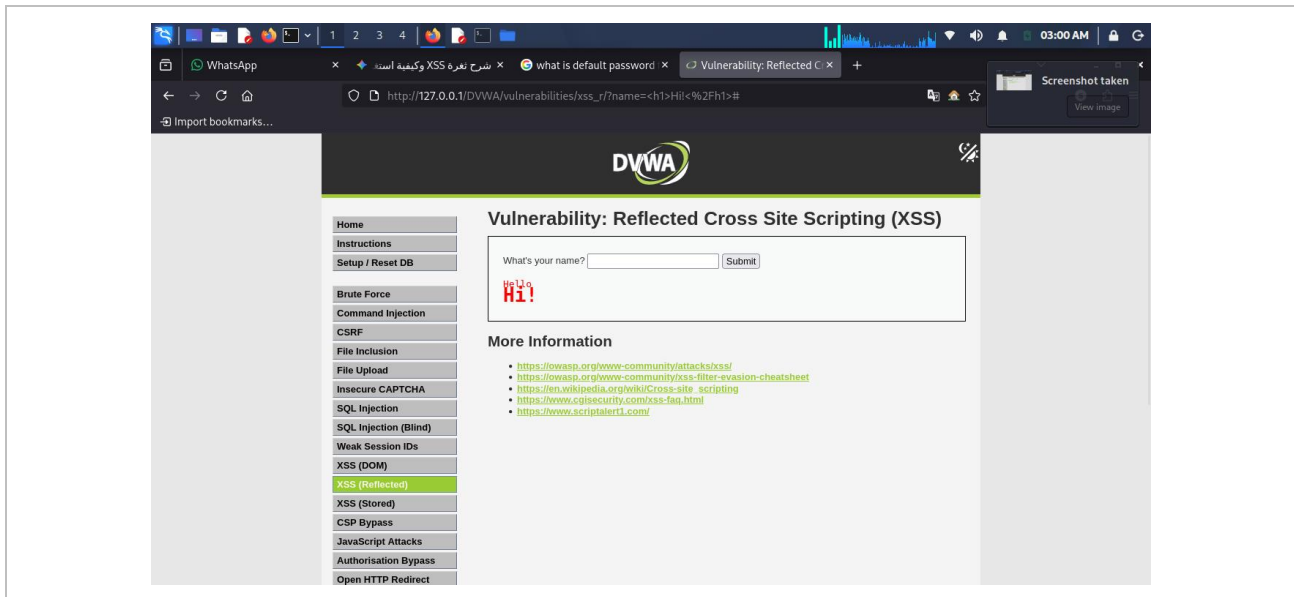


Figure 1.2 – The payload is reflected and rendered unescaped on the page, confirming raw HTML injection.

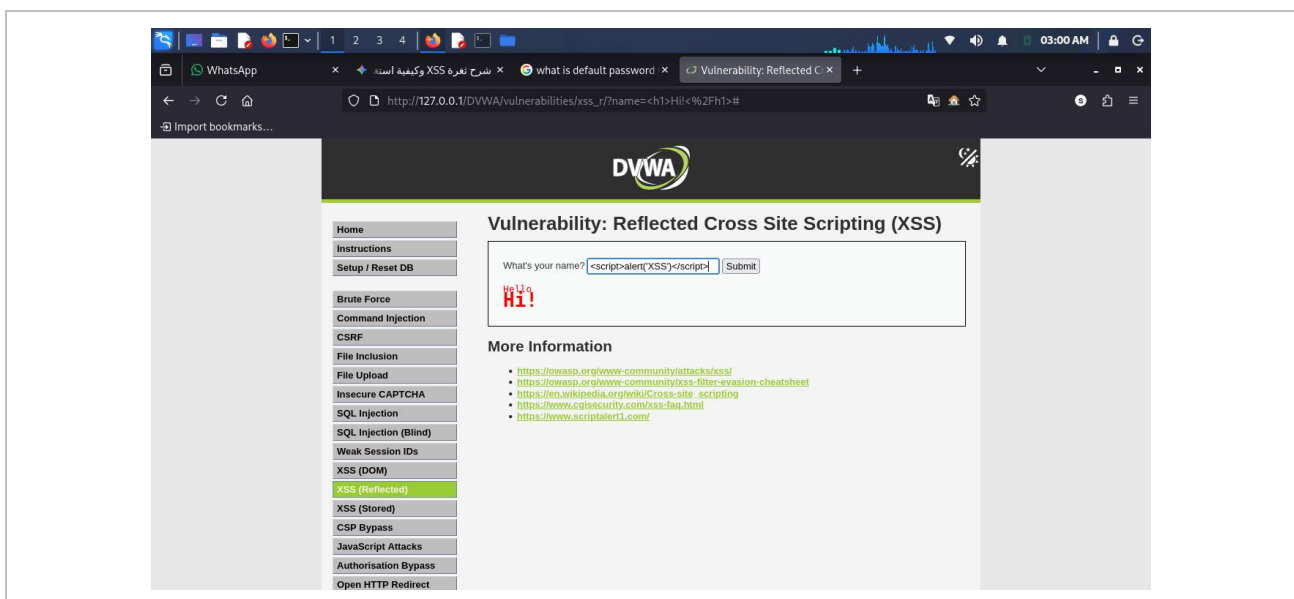


Figure 1.3 – Entering the `<script>alert('XSS')</script>` payload into the input field.

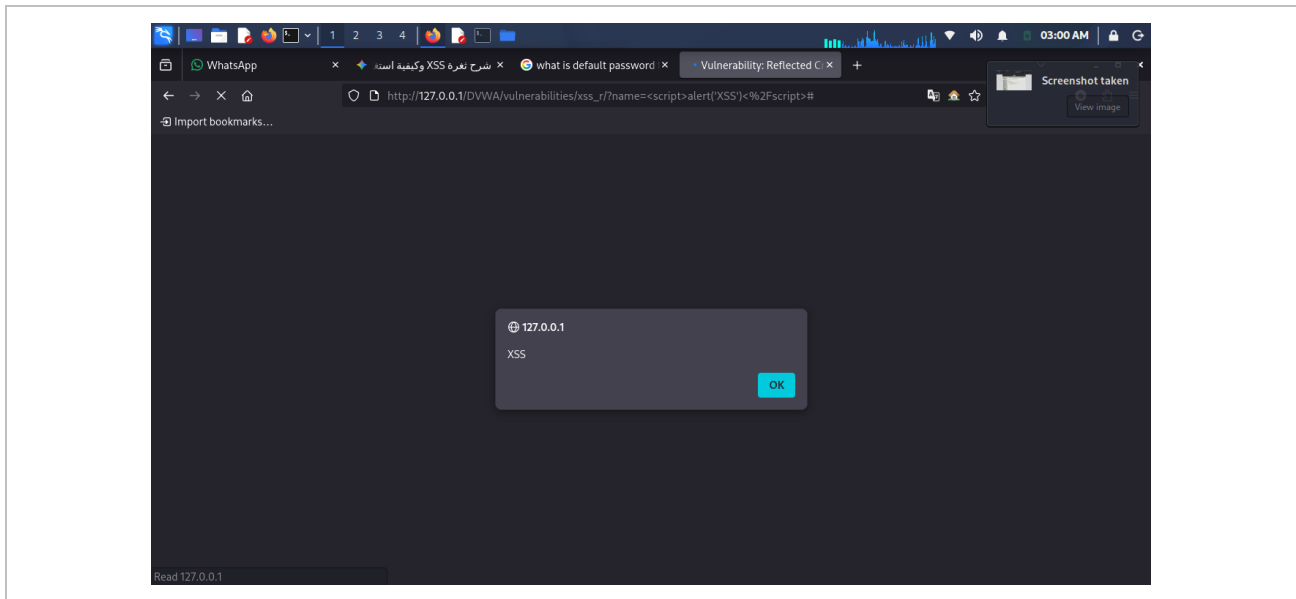


Figure 1.4 – Payload executes successfully at Low security level; JavaScript alert box confirms the XSS.

Finding (Medium): The server attempts to block basic XSS by stripping <script> tags. Bypassed using alternative tags like .

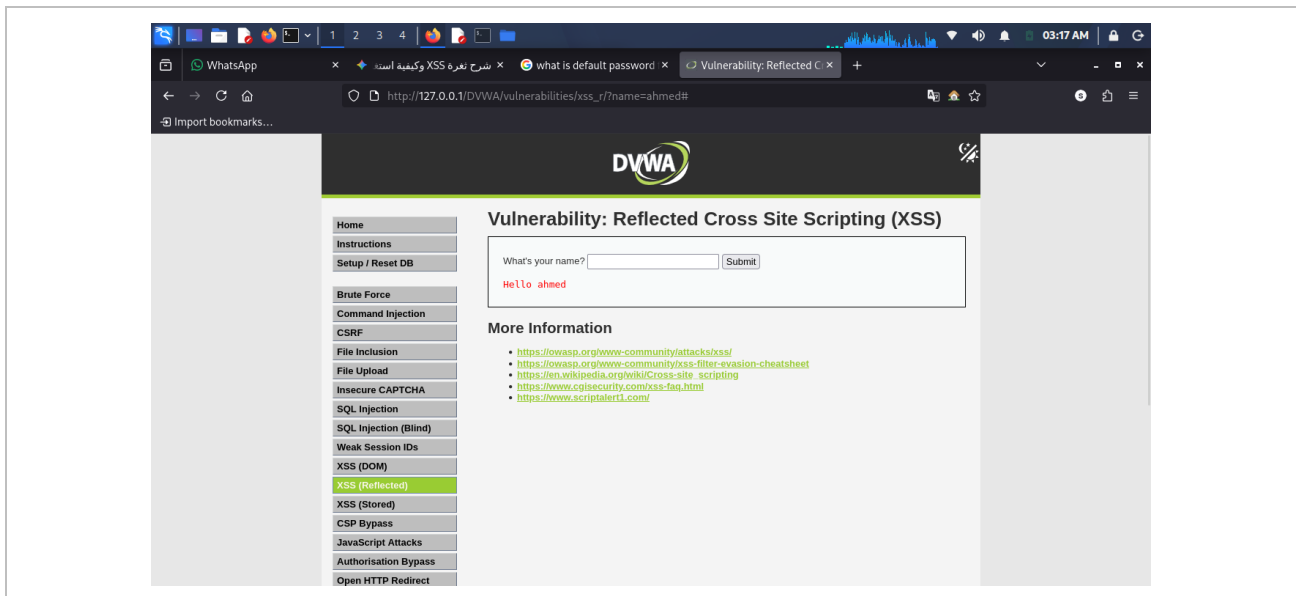


Figure 1.5 – Baseline test with normal input ("ahmed") at Medium security level.

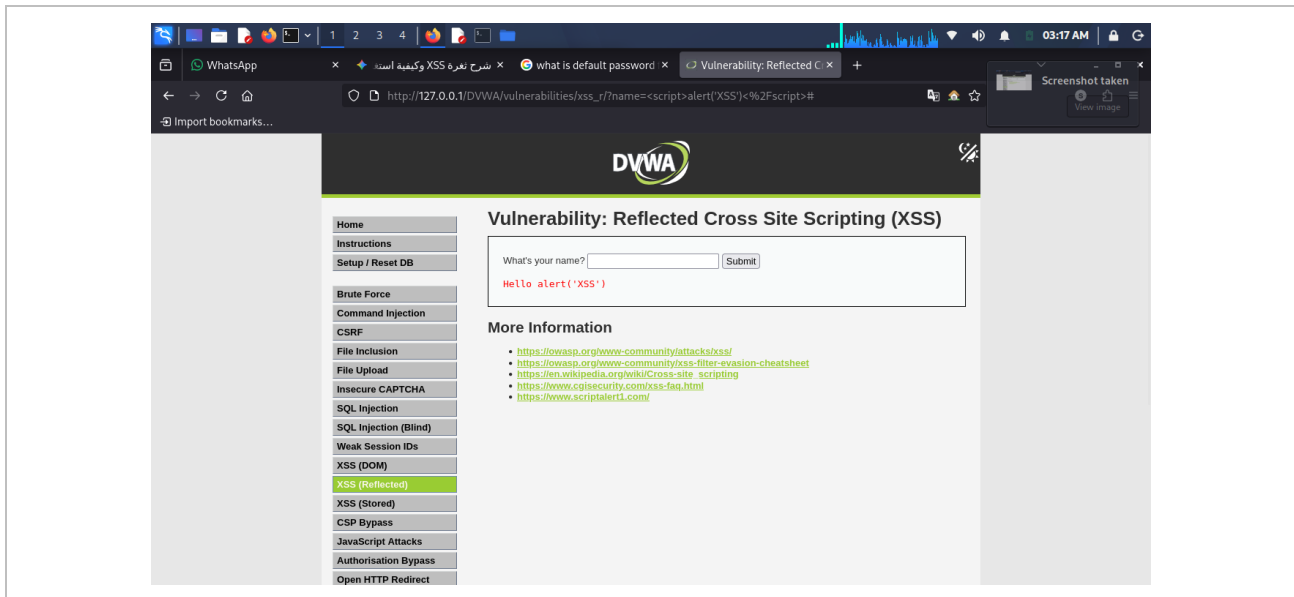


Figure 1.6 – The `<script>` tag is stripped by the server-side filter; only the literal text remains, confirming the filter is active.

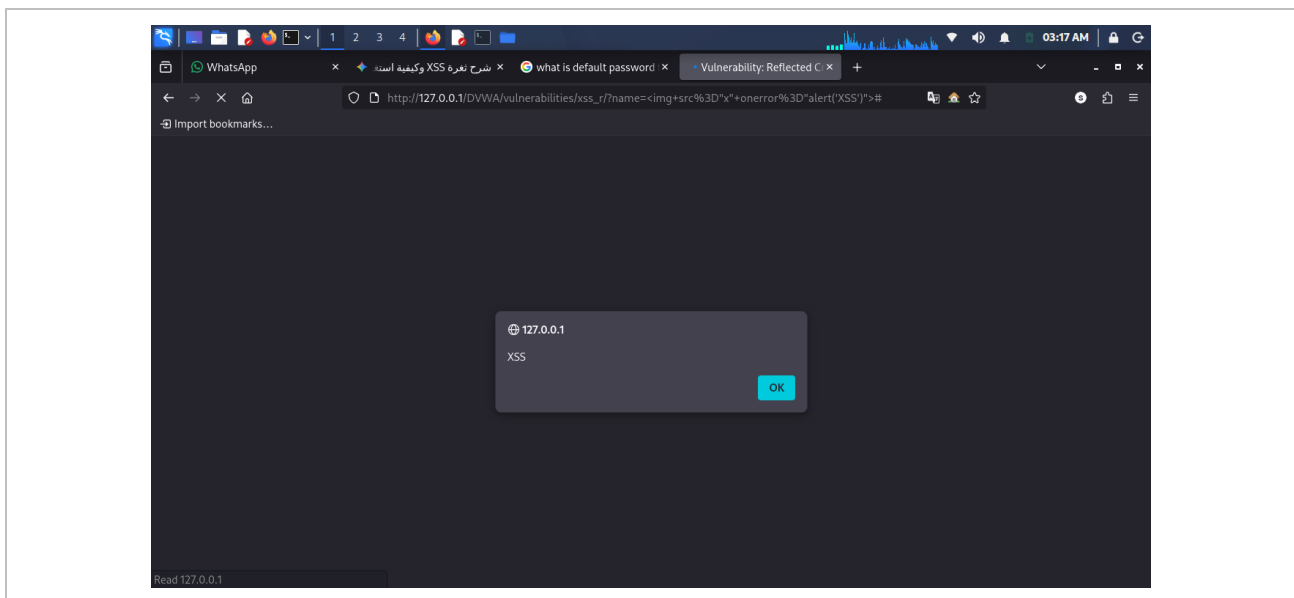


Figure 1.7 – Filter bypassed using ``; the alert box confirms successful bypass.

Remediation: Do not rely on simple string filtering. Apply strong output encoding functions like `htmlspecialchars()` in PHP.

2. Stored Cross-Site Scripting (XSS)

Severity / Level: Low & Medium

Description: The application saves user input directly into the database and displays it to visitors without checking or escaping content.

Finding (Low): Text entered into Guestbook (Message field) is saved directly. Script executes every time any user visits the page.

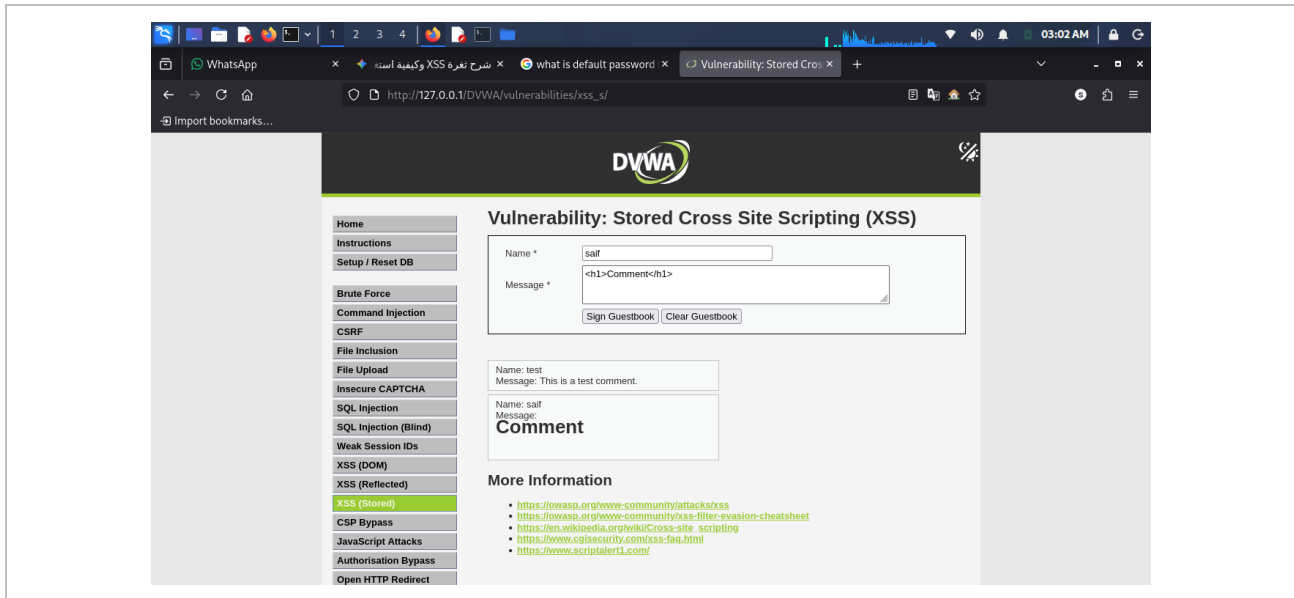


Figure 2.1 – Baseline HTML injection test (<h1>Comment</h1>) submitted; the payload is rendered alongside the default guestbook entry.

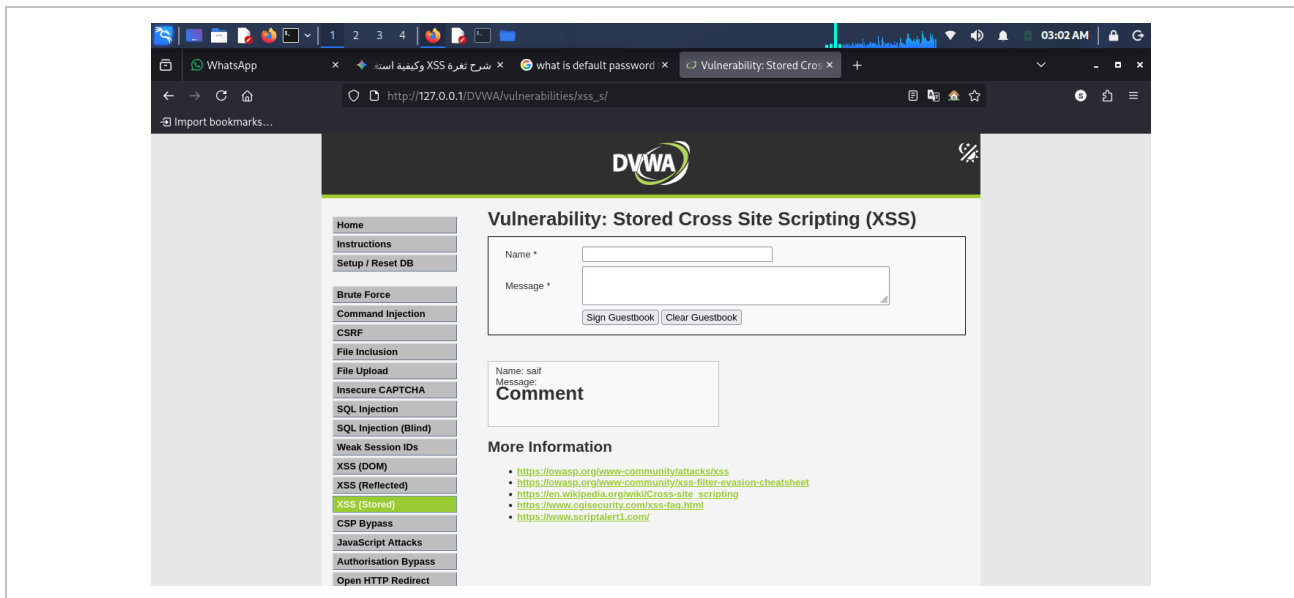


Figure 2.2 – The input form is cleared, while the previously submitted entry remains listed, confirming it was persisted to the database.

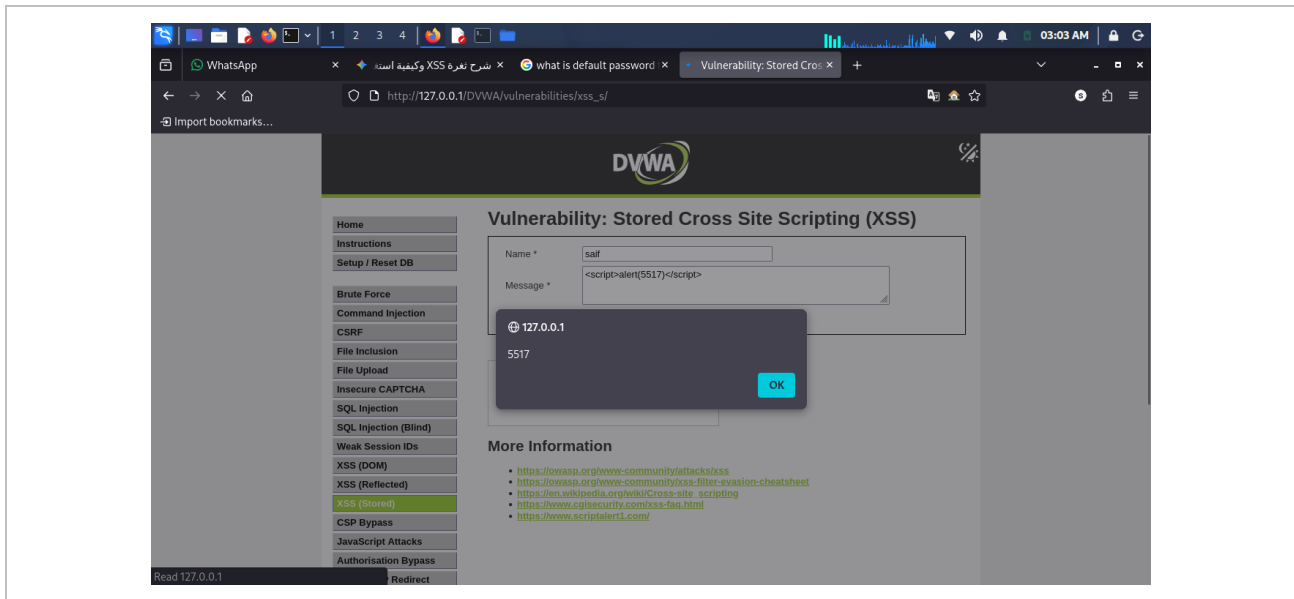


Figure 2.3 – A JavaScript payload `<script>alert(5517)</script>` is submitted and executes immediately upon signing the guestbook.

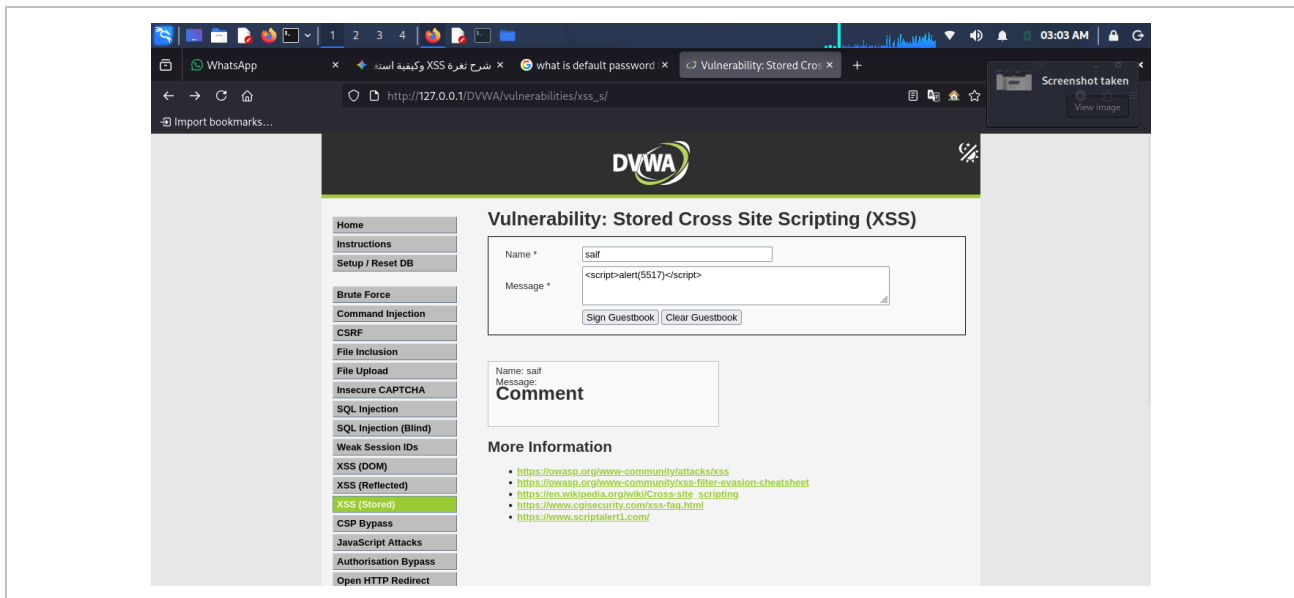


Figure 2.4 – After dismissing the alert, the payload is now embedded in the stored guestbook data.

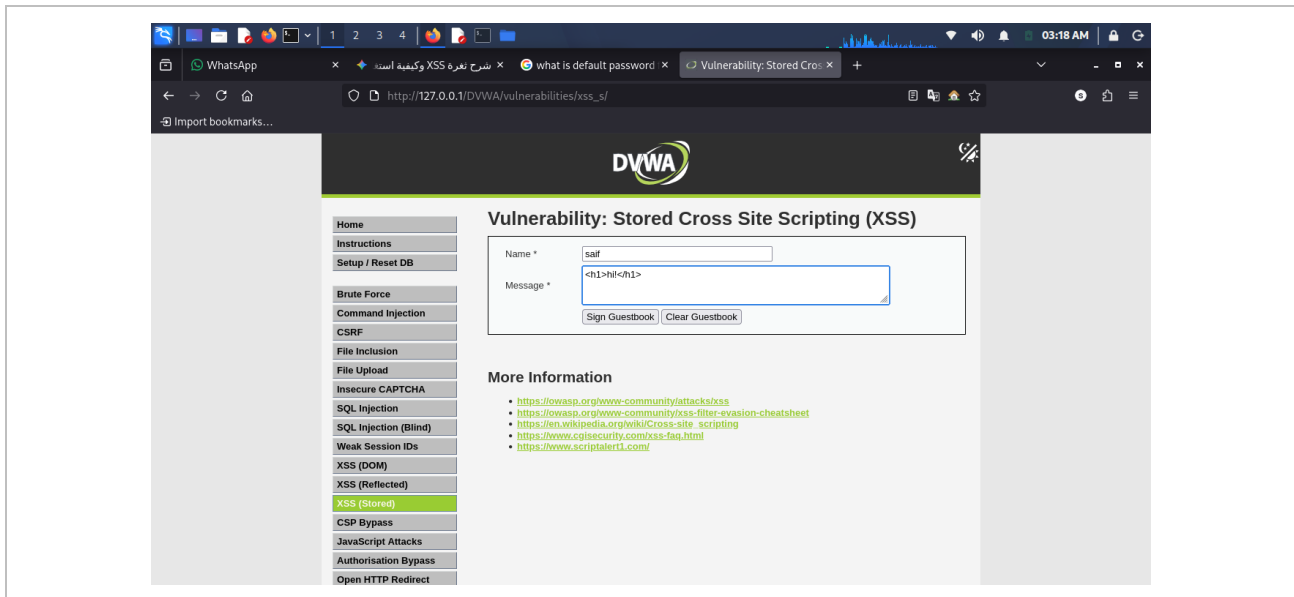


Figure 2.5 – Crafting an additional `<h1>hi</h1>` payload in the Message field.

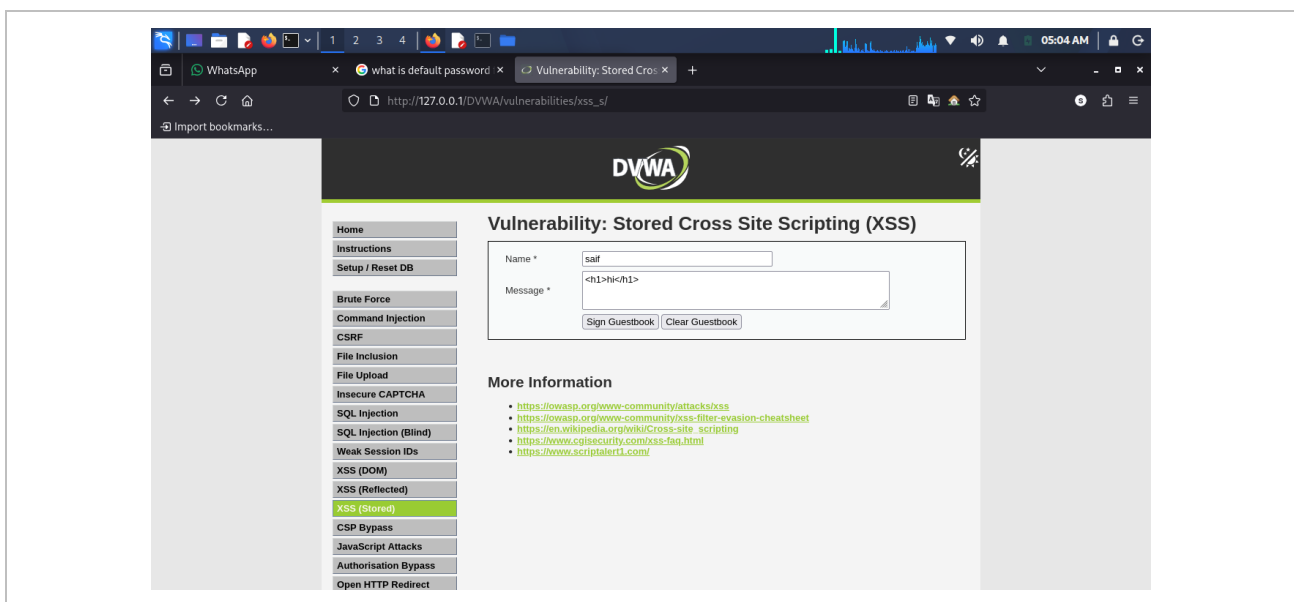


Figure 2.6 – Payload submitted to the guestbook.

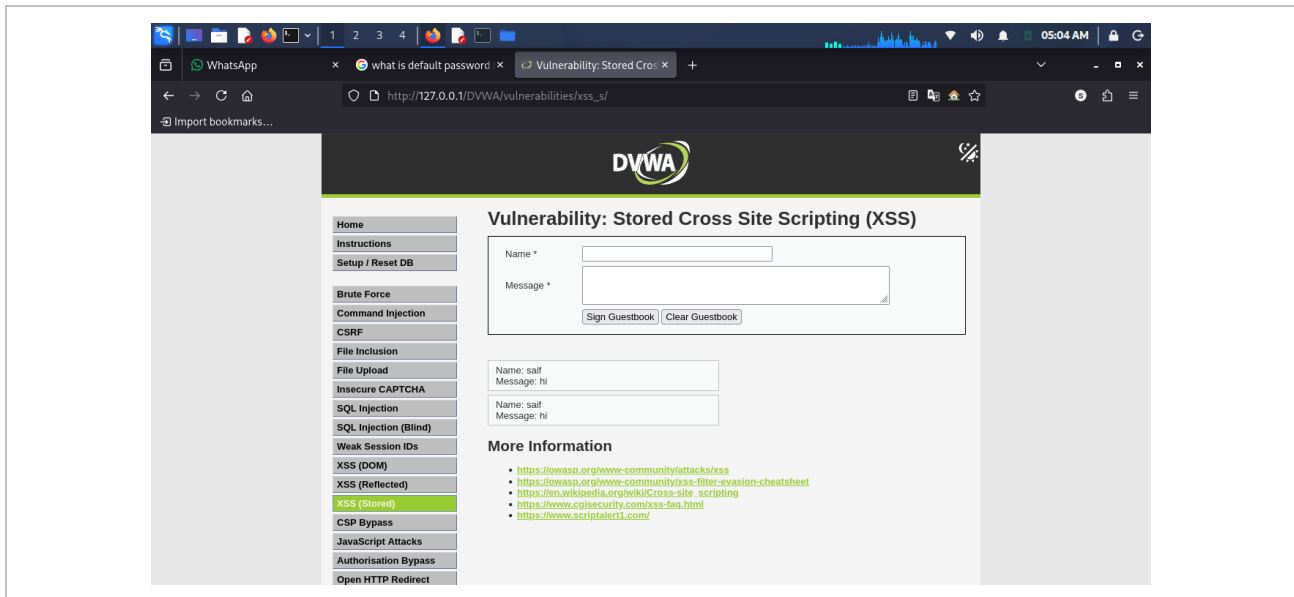


Figure 2.7 – Stored payload persists and renders identically on every page reload, confirming the Stored XSS.

Finding (Medium): Restricts input length on Name field using `maxlength="10"`. Bypassed by editing the DOM using Inspector to submit longer payloads.

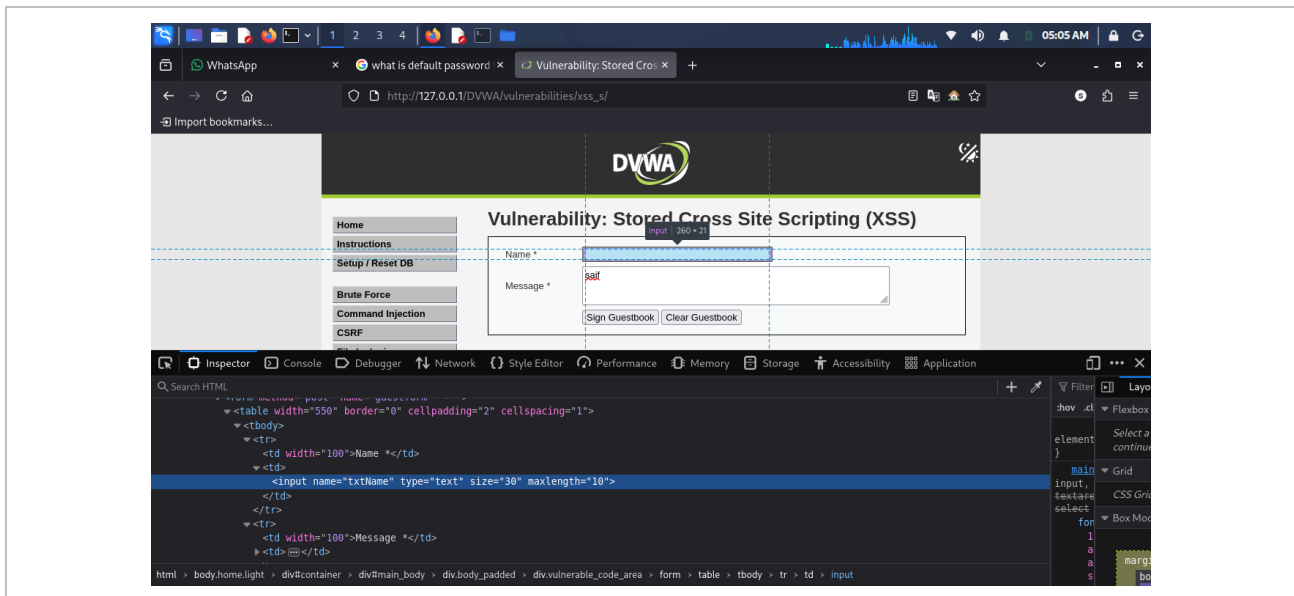


Figure 2.8 – Browser DevTools reveal the client-side restriction `maxlength="10"` on the Name field.

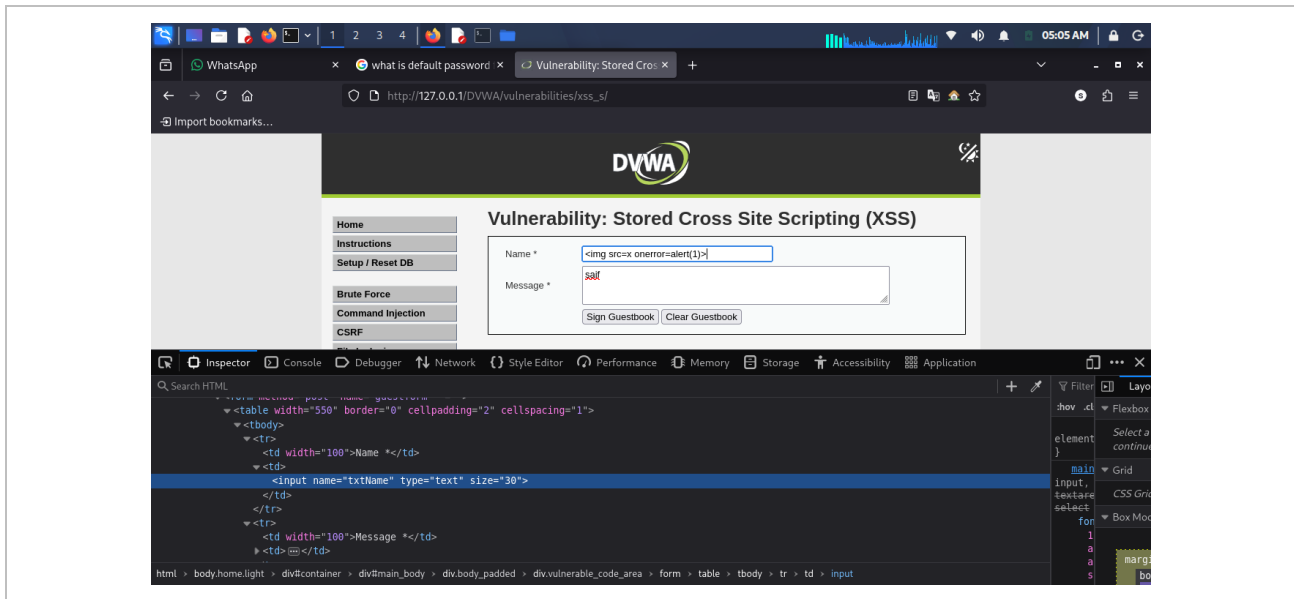


Figure 2.9 – The `maxlength` attribute is removed via DOM editing, allowing the longer payload `<img src=x onerror=alert(1)>` to be entered.

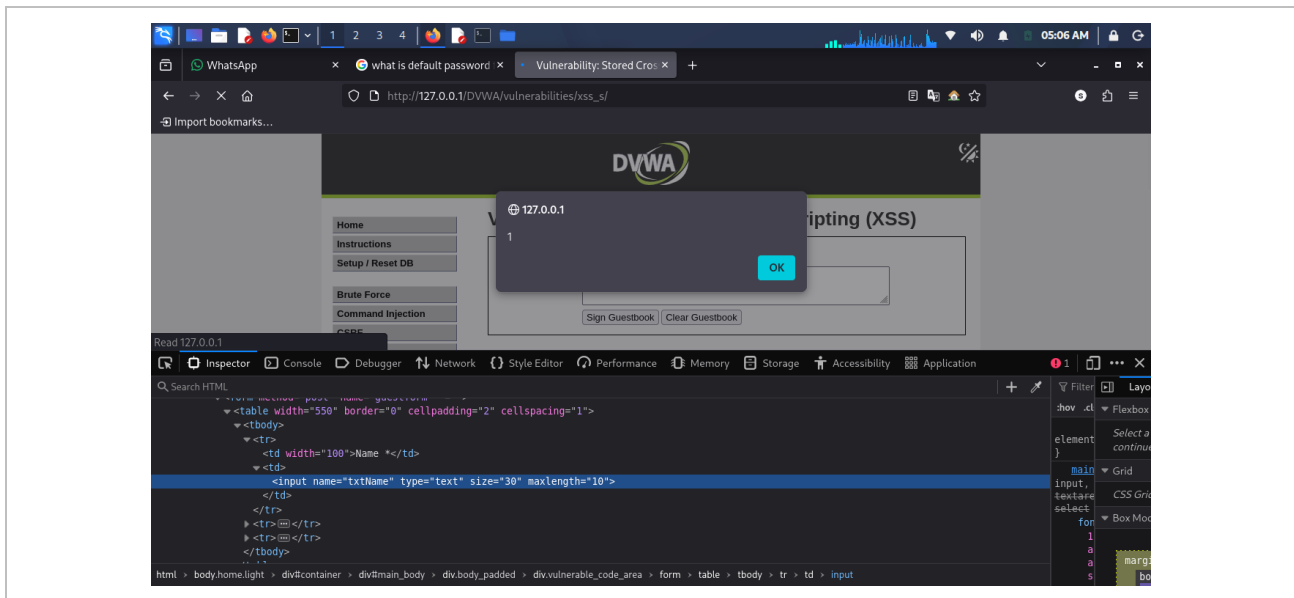


Figure 2.10 – The client-side restriction is bypassed and the injected payload executes successfully.

Remediation: Always validate character length limits on the server side. Encode all stored data before rendering.

3. DOM-Based Cross-Site Scripting (XSS)

Severity / Level: Low

Description: The vulnerability exists entirely within the client-side JavaScript code rather than server-side scripts.

Finding (Low): Reads default parameter directly from URL and writes into DOM without checking:
?default=<script>alert(5517)</script>

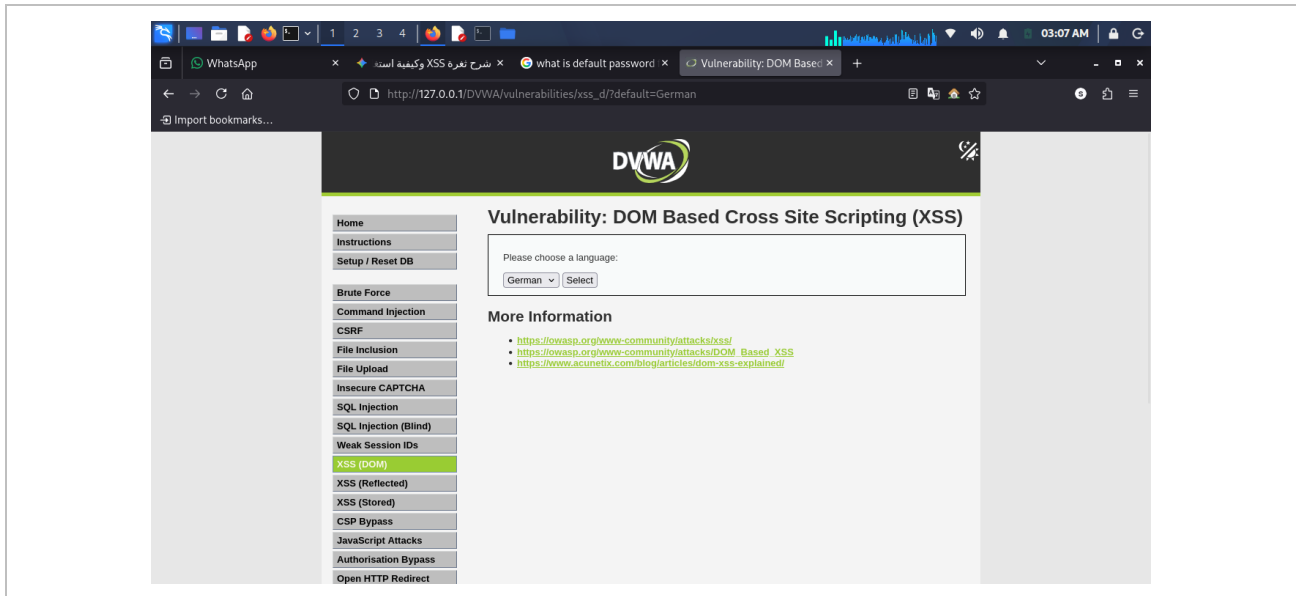


Figure 3.1 – Baseline behavior: selecting "German" from the language dropdown.

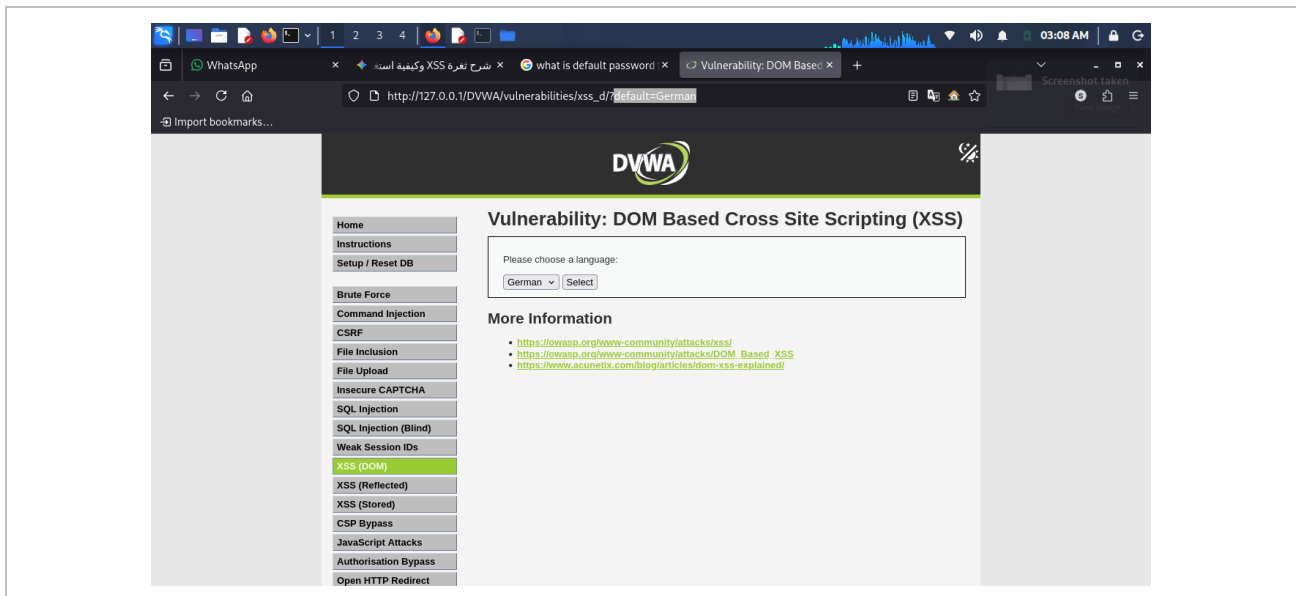


Figure 3.2 – The default parameter in the URL is identified as the source controlling the DOM output.

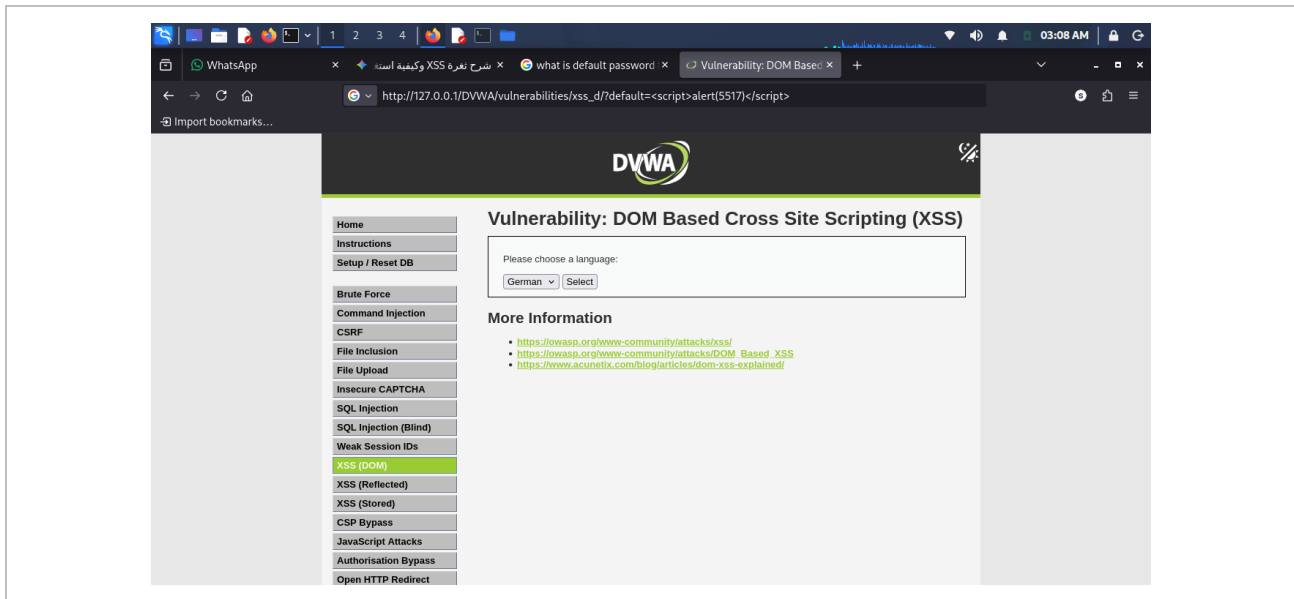


Figure 3.3 – The parameter value is replaced with the payload `?default=<script>alert(5517)</script>` in the address bar.

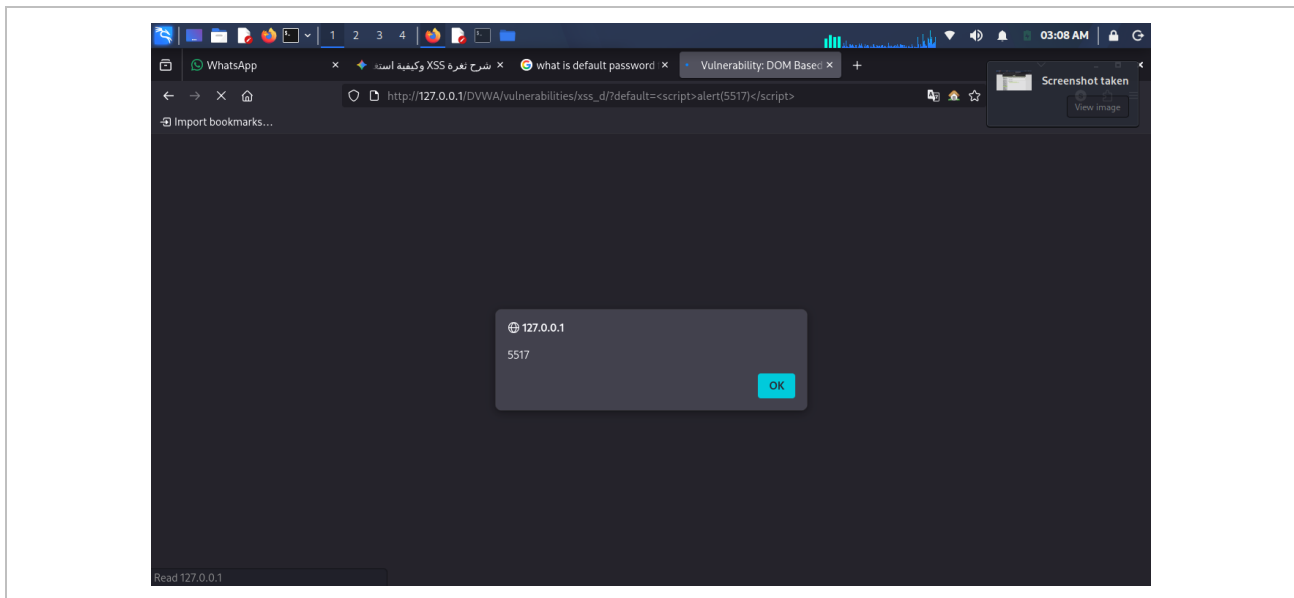


Figure 3.4 – The script executes purely on the client side, confirming the DOM-based XSS.

Remediation: Avoid unsafe JavaScript methods like `document.write()`. Use `textContent` or `innerText`.

4. SQL Injection - Blind (Time-Based)

Severity / Level: Medium

Description: Uses drop-down menu for User ID options without displaying detailed database error messages.

Finding (Medium): Inspecting HTML source code allows modifying the `<option value="...">` attribute to: `1 and sleep(5)#` which delays response by 5 seconds.

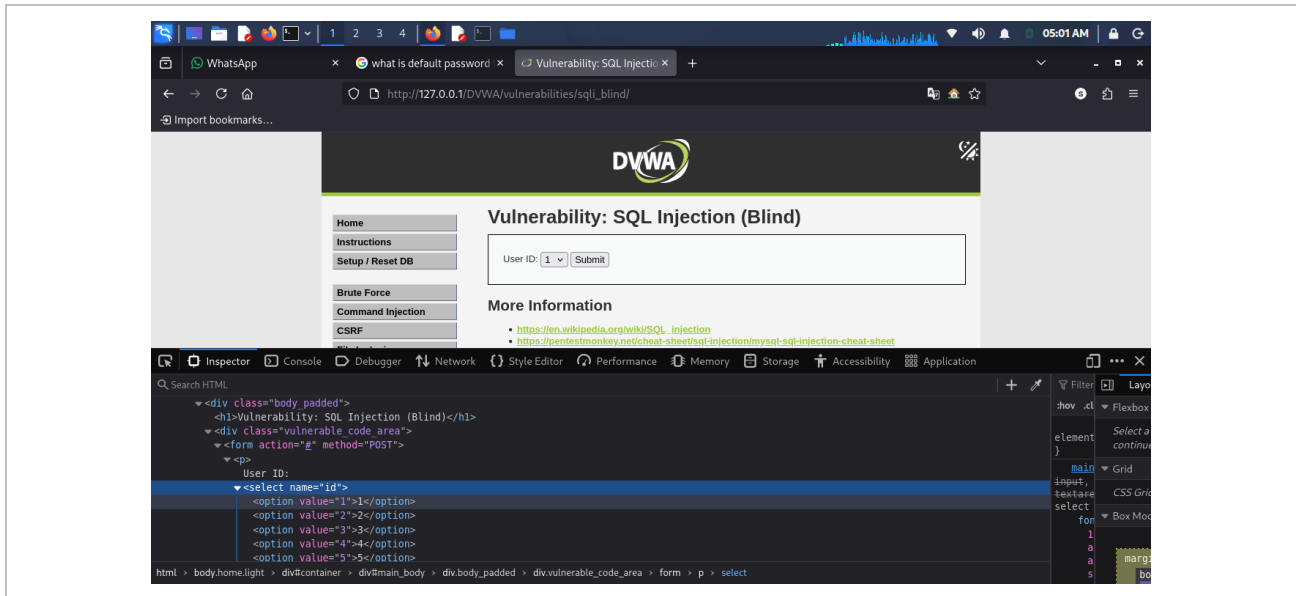


Figure 4.1 – Baseline inspection of the User ID `<select>` element and its default option values (1–5).

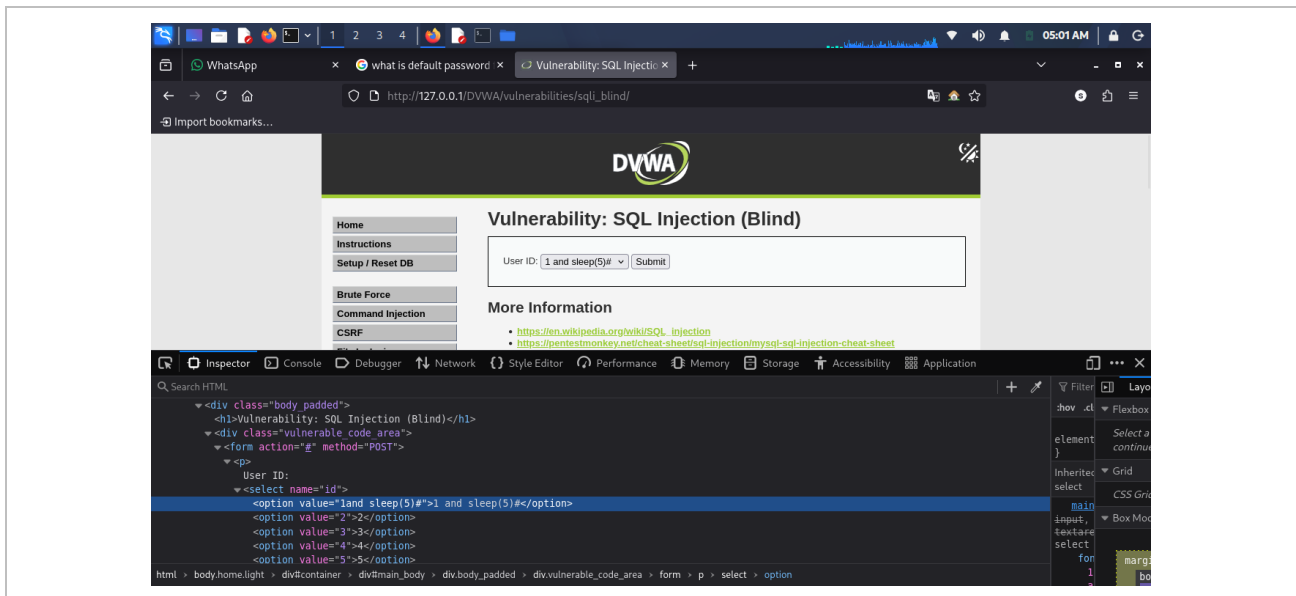


Figure 4.2 – The option value is edited via DevTools to inject the payload: `1 and sleep(5)#`.

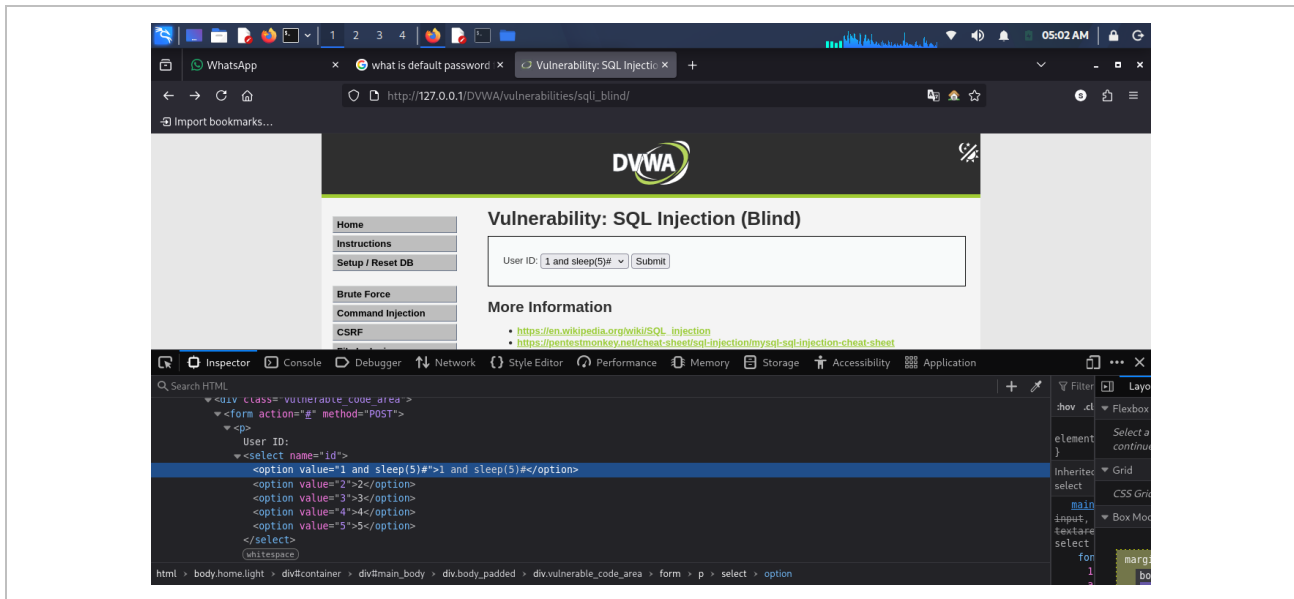


Figure 4.3 – The modified option is selected and ready to be submitted.

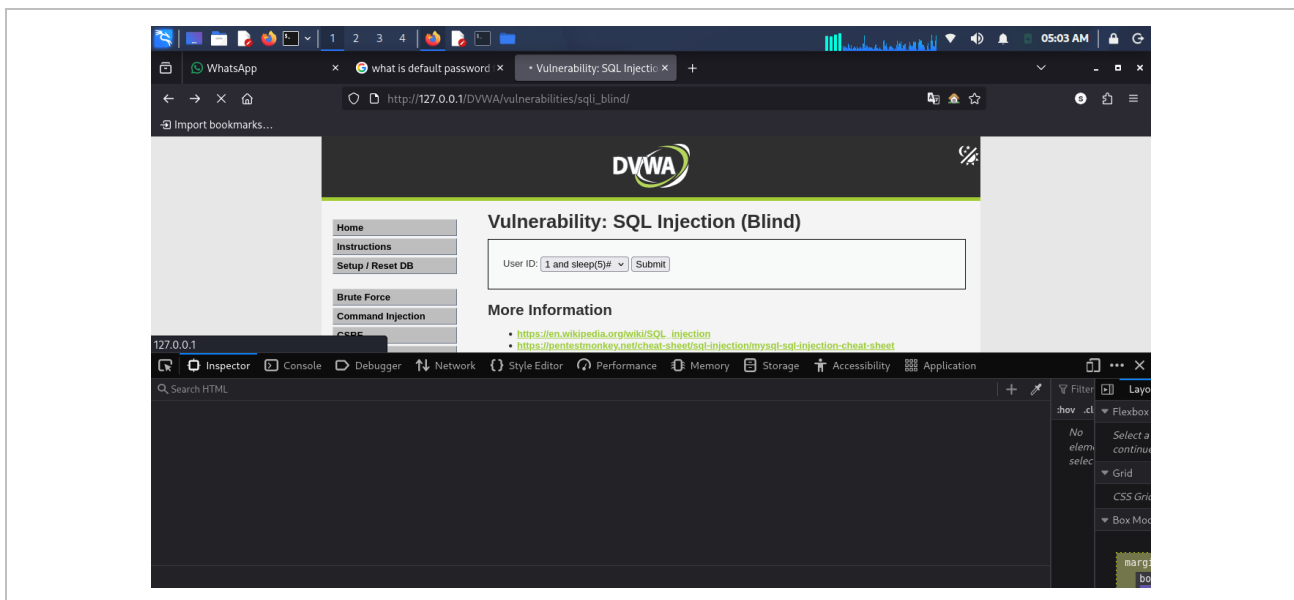


Figure 4.4 – The page hangs while the injected SQL query executes the SLEEP(5) delay.

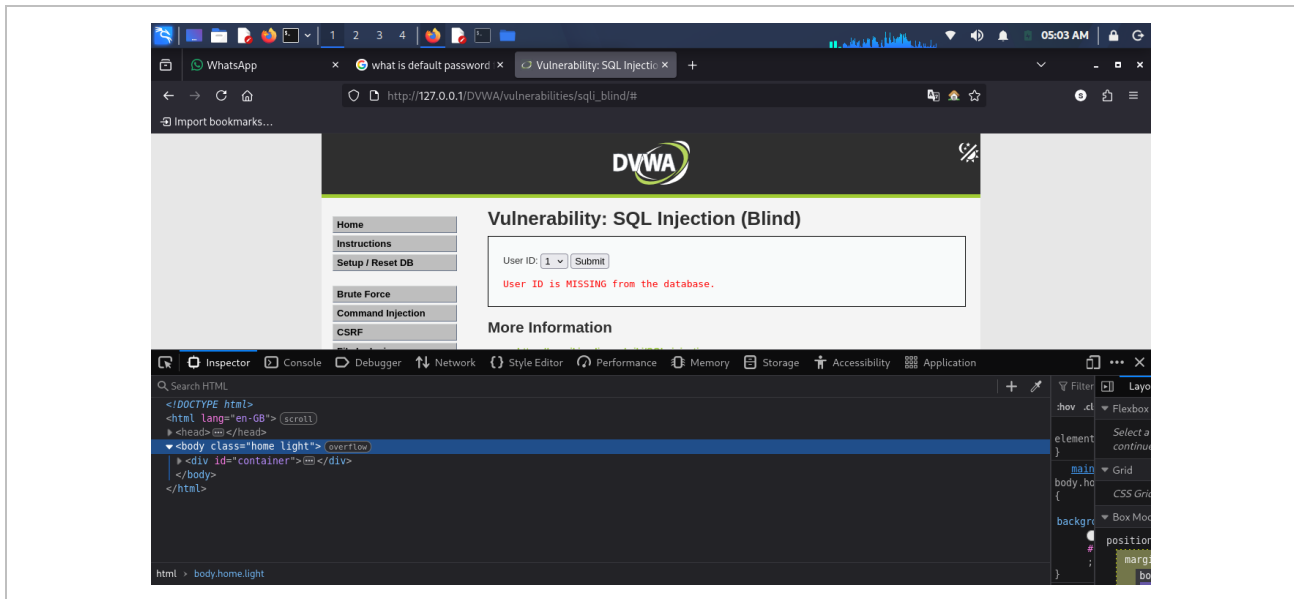


Figure 4.5 – The response returns after a ~5 second delay, confirming the time-based Blind SQL Injection.

Remediation: Use Prepared Statements (Parameterized Queries) for all database operations.

Remediation Checklist

- Implement Prepared Statements for all database interactions.
- Apply Context-Aware Output Encoding before displaying input on web pages.
- Enforce Server-Side Validation instead of relying on browser-based controls.
- Implement Content Security Policy (CSP) headers.